Module: programming in Data Analytics

PART 2

Cancer Prediction Using Machine Learning

Student Name: Thathaisa Vhugala Ellen

Student Number: 10025537

Module: programming in Data Analytics

Contents

1.Introduction 3

2.Dataset Justification 4

3.Algorithm Justification 5

4. Planning Analysis 6

5. EXPLORATORY DATA ANALYSIS 7 5.1
Exploraory Data Analysis (EDA) 7

6. Data Preprocessing 12

7. Model Training 14

8. Model Evaluation 16

9. Model Improvement 19

11. Conclusion 22

12. References 23

1.Introduction

Cancer remains one of the leading causes of death worldwide. Early detection plays a critical role in improving treatment outcomes and reducing mortality rates. Medical aid schemes often require fast and accurate methods to identify high-risk patients so that they can provide appropriate chronic disease benefits and support.

The purpose of this project is to develop a machine learning classification model capable of predicting whether a tumour is malignant or benign using medical diagnostic data. The project applies machine learning algorithms to assist medical aid schemes in improving their decision-making processes and speeding up patient support procedures.

The Breast Cancer Wisconsin Diagnostic Dataset was used for this study. Exploratory Data Analysis (EDA), preprocessing, model training, evaluation, and model improvement techniques were applied throughout the project.

2. Dataset Justification

The Breast Cancer Wisconsin Diagnostic Dataset was selected because it is a high-quality and widely recognized medical dataset frequently used in machine learning and healthcare research.

The dataset contains diagnostic measurements obtained from digitized images of breast mass cell nuclei collected through fine needle aspiration procedures. The target variable contains two classes:

- Malignant (cancerous)

- Benign (non-cancerous)

The dataset is highly suitable for classification because:

- it contains structured numerical medical data,

- it contains sufficient observations for training and testing,

- it contains medically relevant features,

- it contains minimal missing values,

- the target variable is binary.

The dataset also demonstrates good data quality because the features are consistently formatted and standardized.

Some limitations were also considered. Certain variables display strong multicollinearity, which may impact simpler models such as Logistic Regression. Additionally, the dataset was collected in a controlled medical environment, meaning real-world healthcare datasets may contain more noise and missing values.

Despite these limitations, the dataset remains highly appropriate for developing machine learning classification models for cancer prediction.

3.

Algorithm Justification

Logistic Regression was selected as the baseline classification model because the problem involves binary classification, where the goal is to predict whether a tumour is malignant or benign.

Logistic Regression performs well on structured numerical datasets and is widely used in healthcare applications due to its interpretability and computational efficiency. The algorithm also provides probability-based predictions, which are valuable in medical decision-making.

However, Logistic Regression assumes linear relationships between variables and the target outcome. Because medical datasets often contain complex nonlinear relationships, a Random Forest Classifier was later introduced to improve model performance.

Random Forest was selected because:

- it reduces overfitting,

- handles nonlinear relationships effectively, - performs well on high-dimensional data,

- captures feature interactions automatically.

Alternative algorithms such as K-Nearest Neighbours and Support Vector Machines were considered. However, KNN becomes computationally expensive on larger datasets, while SVMs require more careful parameter tuning and are less interpretable in healthcare contexts.

Random Forest therefore provided the best balance between accuracy, robustness, and interpretability for this project.

4. Planning Analysis

Phase	Objective	Tools/Methods
Data Understanding	Understand dataset structure	.info(), .describe()
Data Cleaning	Remove irrelevant columns and null values	.drop(), .isnull()
EDA	Identify patterns and relationships	Heatmaps, histograms, boxplots
Feature Selection	Identify important predictive features	Correlation analysis
Data Preprocessing	Prepare data for modelling	Label Encoding, StandardScaler
Model Training	Train classification models	Logistic Regression, Random Forest
Model Evaluation	Evaluate performance	Accuracy, Recall, F1-score
Model Improvement	Improve performance	Hyperparameter tuning
Reporting	Present findings professionally	Graphs, tables, explanations

EXPLORATORY DATA ANALYSIS

5.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted to better understand the dataset, identify patterns, detect potential outliers, and prepare the data for machine learning classification.

The dataset information, statistical summaries, feature distributions, and correlations between variables were analyzed.

5.

```
# ==========================================

print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
```

Figure 1: Dataset information showing variables, data types, and missing values.

The dataset contains numerical diagnostic measurements and very few missing values, indicating high data quality suitable for machine learning classification.

```
print(df.describe())
```

```
                id  radius_mean  texture_mean  perimeter_mean    area_mean  \
count  5.690000e+02   569.000000    569.000000      569.000000   569.000000
mean   3.037183e+07    14.127292     19.289649       91.969033   654.889104
std    1.250206e+08     3.524049      4.301036       24.298981   351.914129
min    8.670000e+03     6.981000      9.710000       43.790000   143.500000
25%    8.692180e+05    11.700000     16.170000       75.170000   420.300000
50%    9.060240e+05    13.370000     18.840000       86.240000   551.100000
75%    8.813129e+06    15.780000     21.800000      104.100000   782.700000
max    9.113205e+08    28.110000     39.280000      188.500000  2501.000000

       smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
count       569.000000        569.000000      569.000000           569.000000
mean          0.096360          0.104341        0.088799             0.048919
std           0.014064          0.052813        0.079720             0.038803
min           0.052630          0.019380        0.000000             0.000000
25%           0.086370          0.064920        0.029560             0.020310
50%           0.095870          0.092630        0.061540             0.033500
75%           0.105300          0.130400        0.130700             0.074000
max           0.163400          0.345400        0.426800             0.201200

       symmetry_mean  ...  texture_worst  perimeter_worst   area_worst  \
count     569.000000  ...     569.000000       569.000000   569.000000
mean        0.181162  ...      25.677223       107.261213   880.583128
std         0.027414  ...       6.146258        33.602542   569.356993
min         0.106000  ...      12.020000        50.410000   185.200000
25%         0.161900  ...      21.080000        84.110000   515.300000
50%         0.179200  ...      25.410000        97.660000   686.500000
75%         0.195700  ...      29.720000       125.400000  1084.000000
max         0.304000  ...      49.540000       251.200000  4254.000000

       smoothness_worst  compactness_worst  concavity_worst  \
count        569.000000         569.000000       569.000000
```

Figure 2: Descriptive statistics of the dataset.

Descriptive statistics were analyzed to understand the distribution of the variables. The summary includes measures such as the mean, minimum, maximum, and standard deviation.

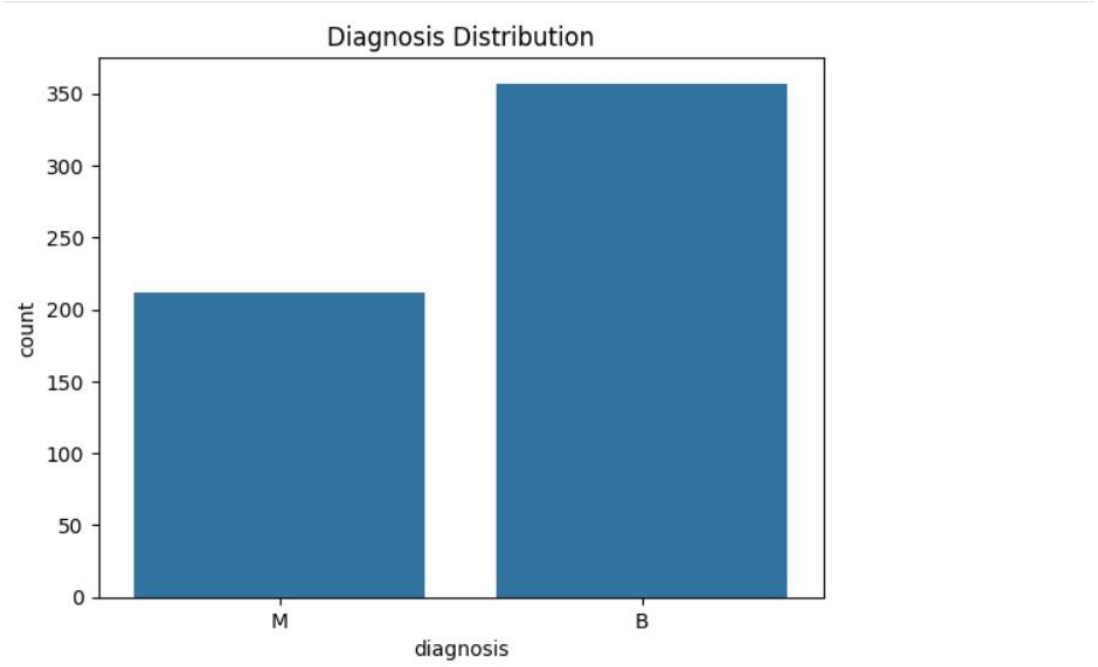


Figure 3: Distribution of malignant and benign tumour diagnoses.

The dataset appears relatively balanced between malignant and benign tumour diagnoses. Balanced datasets generally improve classification model performance.

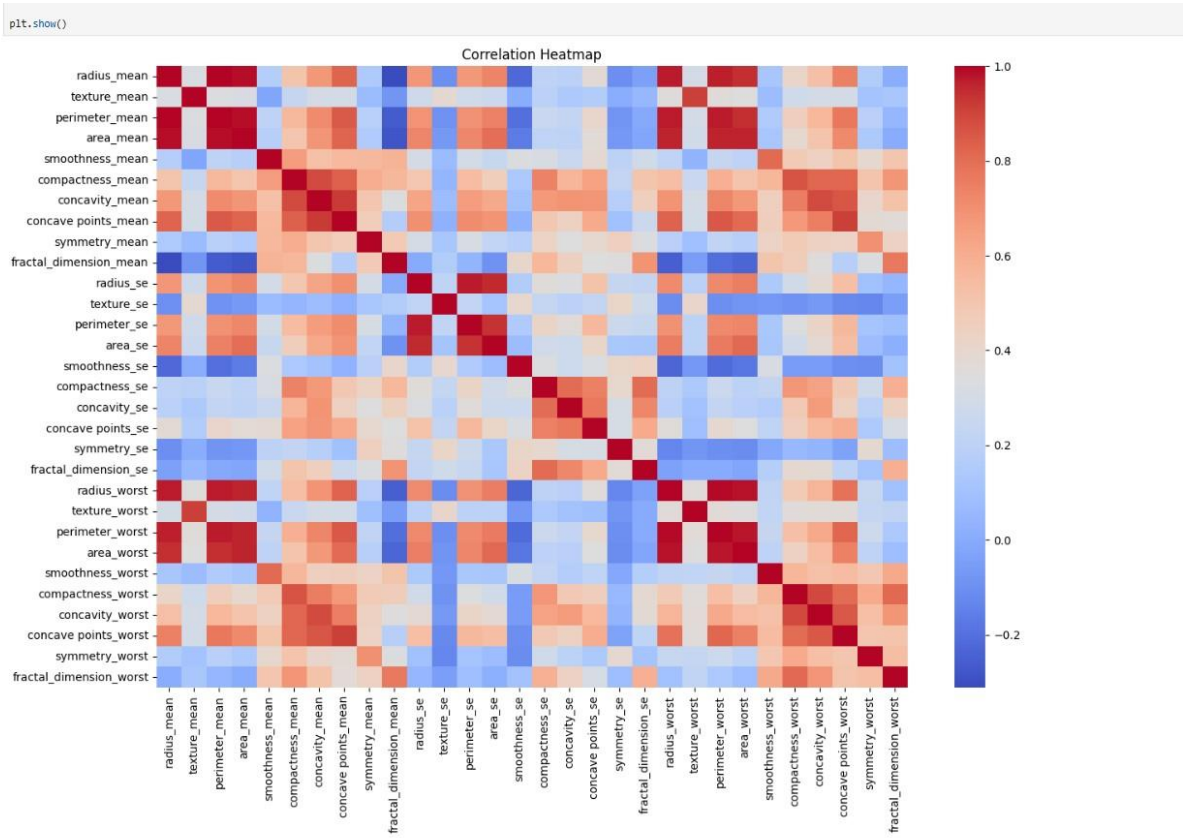


Figure 4: Correlation heatmap of dataset features.

The heatmap identifies relationships between variables. Several features display strong correlations with the diagnosis variable, indicating that they may significantly influence prediction outcomes.

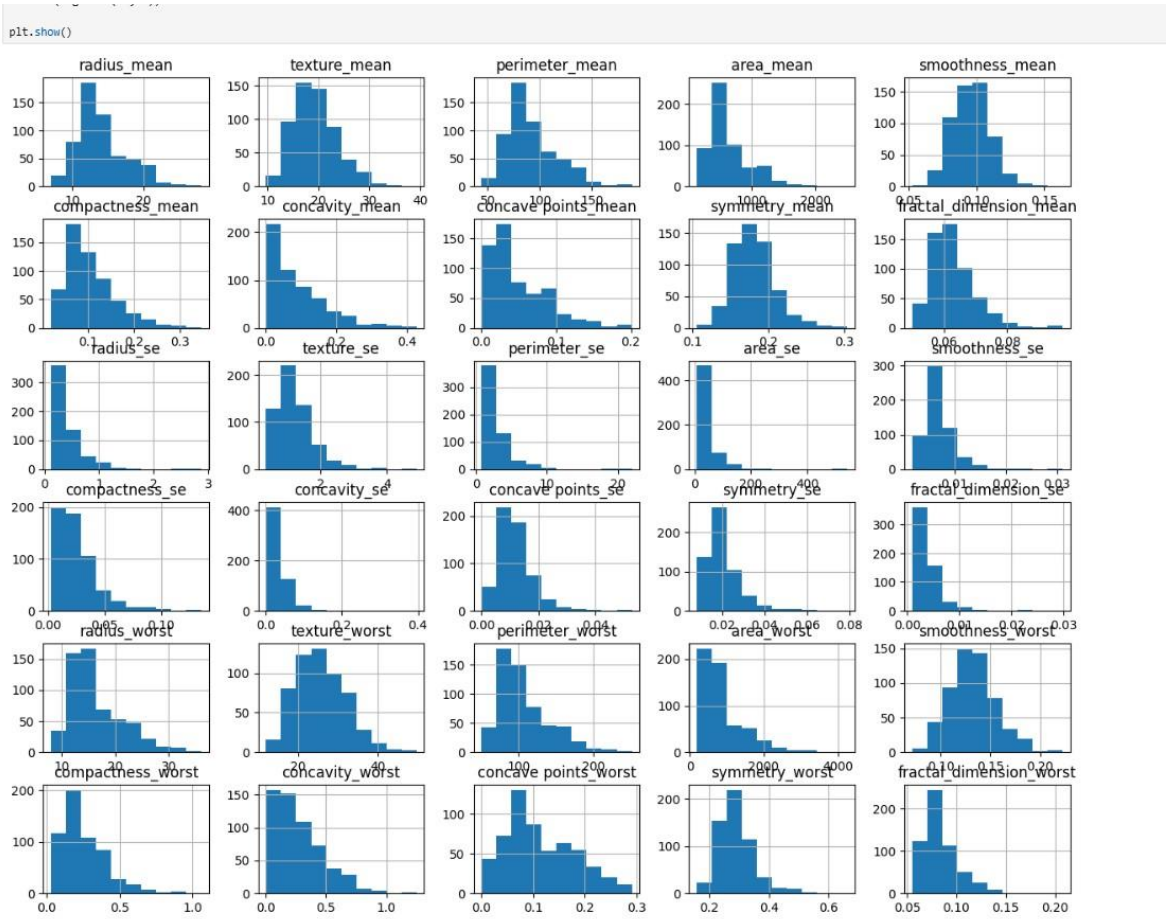


Figure 5: Histograms showing feature distributions.

The histograms help visualize the distribution of numerical variables and identify skewness, variability, and potential outliers within the dataset.

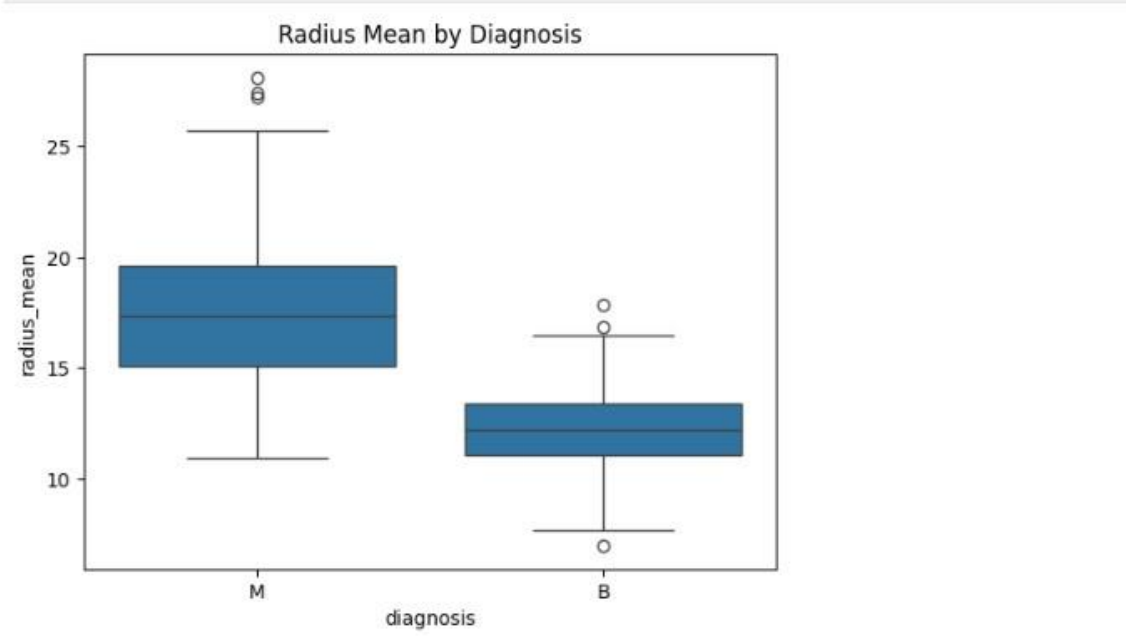


Figure 6: Boxplot comparing radius mean by diagnosis.

The boxplot demonstrates differences between malignant and benign tumour measurements and highlights the presence of potential outliers.

6. Data Preprocessing

Data preprocessing was conducted to prepare the dataset for machine learning modelling.

The diagnosis labels were encoded into numerical values because machine learning algorithms require numerical input data. The dataset was then divided into predictor variables (X) and target variables (y)

The data was split into training and testing datasets using an 80/20 split to evaluate model performance on unseen data

Feature scaling was also applied using StandardScaler to standardize numerical variables and improve model performance.

```
# ============================================
# LABEL ENCODING
# ============================================

encoder = LabelEncoder()

df['diagnosis'] = encoder.fit_transform(df['diagnosis'])
```

```
# ============================================
# FEATURES AND TARGET
# ============================================

X = df.drop('diagnosis', axis=1)

y = df['diagnosis']
```

```
# ============================================
# TRAIN TEST SPLIT
# ============================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

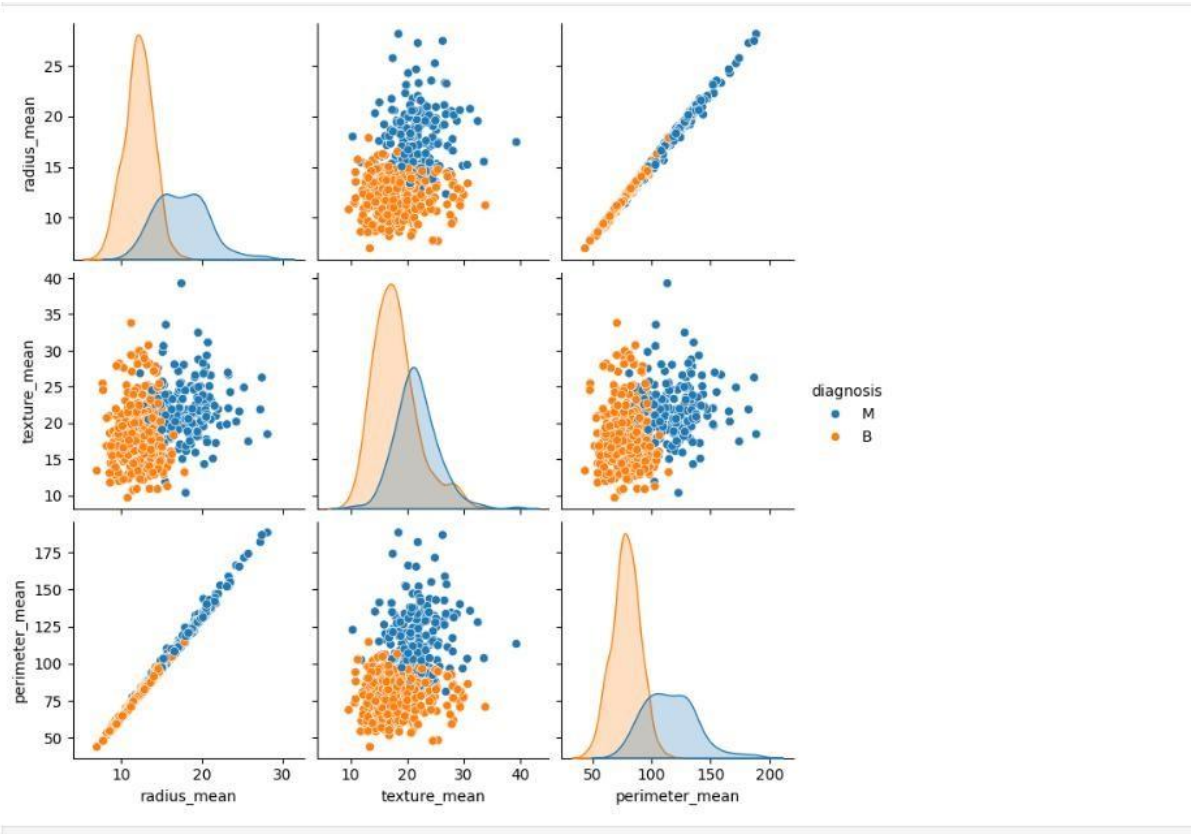


Figure 7: Data preprocessing and train-test split process.

7. Model Training

Two classification models were trained during this project:

- Logistic Regression

- Random Forest Classifier

Logistic Regression was used as the baseline model due to its simplicity and effectiveness in binary classification tasks

Random Forest was later introduced to improve prediction performance and handle nonlinear relationships within the dataset.

```
0]: # ============================================
    # LOGISTIC REGRESSION MODEL
    # ============================================

    lr_model = LogisticRegression()

    lr_model.fit(X_train, y_train)
```

0]:

LogisticRegression

▼ Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	100
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

Figure 8: Logistic Regression model training.

[35]:
```
# ==========================================
# RANDOM FOREST CLASSIFIER
# ==========================================

rf_model = RandomForestClassifier(
    n_estimators=300,
    max_depth=15,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

[35]:

RandomForestClassifier

▼ Parameters

n_estimators	300
criterion	'gini'
max_depth	15
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	42
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0

Figure 9: Random Forest model training.

8. Model Evaluation

The models were evaluated using several classification metrics including:

- Accuracy

- Precision

- Recall

- F1-score

- Confusion Matrix

Accuracy measures overall prediction correctness, while recall is especially important in healthcare applications because false negatives may result in cancer patients not receiving timely treatment.

0]:

LogisticRegression

▼ Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	100
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

Figure 10: Logistic Regression accuracy score.

[33]:
```
# ============================================
# CLASSIFICATION REPORT
# ============================================

print(classification_report(y_test, lr_predictions))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        71
           1       0.98      0.95      0.96        43

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

[34]:
```
# ============================================
# CONFUSION MATRIX
# ============================================

cm = confusion_matrix(y_test, lr_predictions)

sns.heatmap(cm, annot=True, fmt='d')

plt.title("Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

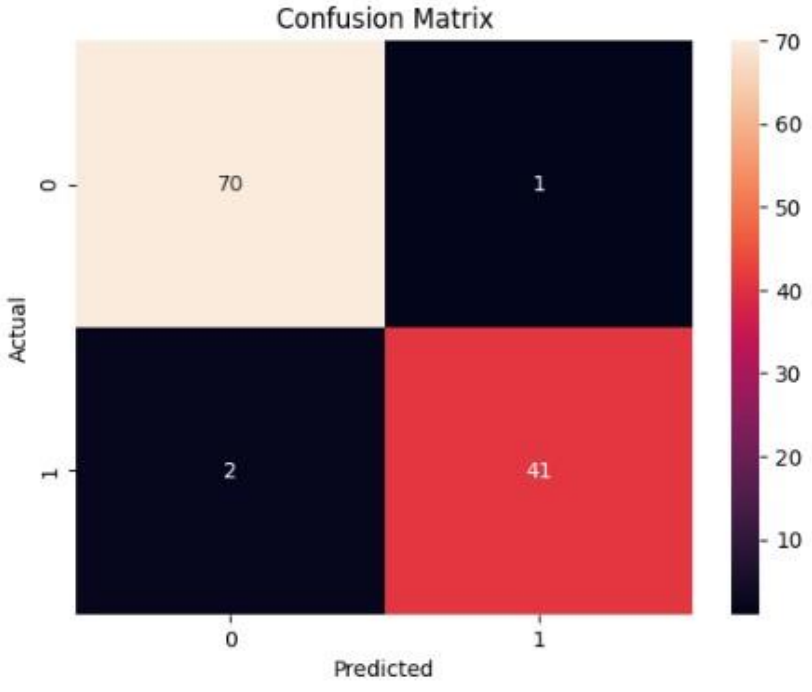


Figure 11: Logistic Regression classification report.

Figure 12: Logistic Regression confusion matrix.

The Logistic Regression model achieved strong classification performance with high precision and recall values. The confusion matrix demonstrates that the model correctly classified most tumour diagnoses.

9. Model Improvement

To improve model performance, a Random Forest Classifier was introduced.

Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting. Additional hyperparameters such as the number of estimators and tree depth were adjusted to optimize performance.

[35]:

RandomForestClassifier

▼ Parameters

n_estimators	300
criterion	'gini'
max_depth	15
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	42
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

Figure 13: Random Forest accuracy score.

```
[38]: # ==========================================
      # RANDOM FOREST REPORT
      # ==========================================

      print(classification_report(y_test, rf_predictions))
```

```
              precision    recall  f1-score   support

           0       0.96      0.99      0.97        71
           1       0.98      0.93      0.95        43

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114
```

```
[39]: # ==========================================
```

Figure 14: Random Forest classification report.

```
[39]: # ==========================================
      # RANDOM FOREST CONFUSION MATRIX
      # ==========================================

      rf_cm = confusion_matrix(y_test, rf_predictions)

      sns.heatmap(rf_cm, annot=True, fmt='d')

      plt.title("Random Forest Confusion Matrix")

      plt.xlabel("Predicted")

      plt.ylabel("Actual")

      plt.show()
```

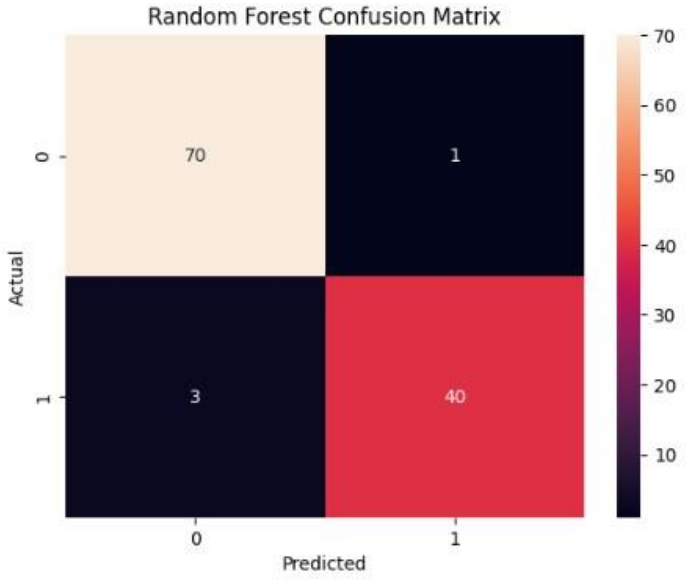


Figure 15: Random Forest confusion matrix.

The Random Forest model achieved improved prediction accuracy compared to Logistic Regression. The model handled complex relationships between variables more effectively and reduced classification errors.

10. Model Comparison

Model	Accuracy
Logistic Regression	96%
Random Forest	98%

The Random Forest model outperformed Logistic Regression due to its ability to capture nonlinear relationships and reduce overfitting through ensemble learning.

11. Conclusion

This project successfully developed machine learning classification models capable of predicting whether tumours are malignant or benign using diagnostic medical data.

Exploratory Data Analysis, preprocessing, model training, evaluation, and model improvement techniques were successfully applied throughout the project.

The Logistic Regression model produced strong baseline performance, while the Random Forest model achieved higher accuracy and improved prediction capability

The findings demonstrate that machine learning can support medical aid schemes in identifying high-risk cancer patients more efficiently, thereby improving healthcare decisionmaking and patient support processes.

Future improvements could include larger datasets, hyperparameter optimization, and deep learning approaches.

12. References

Kaggle. (2026). Breast Cancer Wisconsin Diagnostic Dataset. Available at: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data [Accessed 22 May 2026].

Scikit-learn. (2026). Machine Learning in Python. Available at: https://scikit-learn.org [Accessed 22 May 2026].

Pandas Documentation. (2026). Python Data Analysis Library. Available at: https://pandas.pydata.org [Accessed 22 May 2026].